

Содержание

1. Глоссарий	4
1.1. API	4
1.2. Компонент	4
1.3. Индекс	4
1.4. Ввод-вывод	4
1.5. Слой	4
2. Архитектура информационных систем	5
2.1. Модуль 1: Архитектурные основы	5
2.1.1. Лекция 1: Введение. Принципы построения приложения	5
2.1.1.1. 1. Компонент как класс (C++)	5
2.1.1.2. Зависимости компонентов и слои	6
2.1.1.3. Продуктивность слоя	7
2.1.1.4. Оптимизация системы требует выбора критерия оптимизации	8
2.1.1.5. Принципы SOLID	8
2.1.1.6. Dependency Injection	8
2.1.1.7. Clean Architecture	8
2.1.2. Лекция 2: Альтернативные архитектурные паттерны	9
2.1.2.1. Архитектура на основе фреймворка (Rails, Django)	9
2.1.2.2. Модель акторов (Erlang/Elixir/Akka)	9
2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)	9
2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC	9
2.1.3.1. REST: стандартный веб-протокол	9
2.1.3.2. gRPC: бинарный протокол, высокая производительность	9
2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket	10
2.1.4.1. GraphQL: клиент запрашивает только нужные данные	10
2.1.4.2. WebSocket: двусторонний канал	10
2.2. Модуль 2: Фундаментальные структуры хранилищ данных	11
2.2.1. Лекция 5: Базовые механизмы хранения данных	11
2.2.1.1. CSV-файлы как минимальная форма персистентности	11
2.2.1.2. CSV с добавлением записей и журналированием изменений (WAL, Write Ahead Log)	11
2.2.1.3. Хеш-индексы как первый уровень ускорения выборки	11
2.2.2. Лекция 6: LSM Дерево	13
2.2.2.1. Общая идея	13
2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)	13
2.2.2.3. Memtable	13
2.2.2.4. Bloom Filter	13
2.2.2.5. Компакция – причины, уровни	14
2.2.2.6. Критерии запуска компакций	14
2.2.2.7. Что происходит при компакциях	15
2.2.2.8. Почему L_0 специальный?	15
2.2.2.9. Tombstones	15
2.2.2.10. Полная структура LSM-дерева	15
2.2.3. Лекция 7: B-дерево: индексирование на диске	17
2.2.3.1. B-фактор и структура узла	17
2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища	19
2.2.4.1. ACID (Atomicity, Consistency, Isolation, Durability)	19
2.2.4.2. Уровни изоляции	19
2.2.4.3. Аналитические базы данных (OLAP) и колоночные хранилища	19
2.2.4.4. Сжатие колонок	19
2.2.4.5. Сравнение подходов	19
2.3. Модуль 3: Другие хранилища данных	19

2.3.1. Лекция 9: Аналитика в оперативной памяти	19
2.3.1.1. Vectorized execution (блочная обработка)	19
2.3.1.2. DuckDB	19
2.3.2. Лекция 10: Кэш в оперативной памяти	19
2.3.2.1. Skip List	19
2.3.2.2. Incremental Rehashing	19
2.3.2.3. HyperLogLog	19
2.3.2.4. Redis	19
2.4. Модуль 4: Методы поиска похожих/близких элементов	19
2.4.1. Лекция 11: Специализированные индексы для поиска	19
2.4.1.1. Inverted Index (перевёрнутые индексы)	19
2.4.1.2. Trie (префиксные деревья)	19
2.4.1.3. Bitmap indexes	19
2.4.2. Лекция 12: Векторные индексы для поиска подобия	19
2.4.2.1. Основы векторных представлений	19
2.4.2.2. HNSW (Hierarchical Navigable Small World)	19
2.4.2.3. LSH (Locality-Sensitive Hashing)	19
2.4.2.4. Примеры: Weaviate, Pinecone, Qdrant	20
2.4.3. Лекция 13: Геопространственные структуры	20
2.4.3.1. R-Tree	20
2.4.3.2. QuadTree и KD-Tree	20
2.5. Модуль 5: Операционные компоненты	20
2.5.1. Лекция 14: Прокси и Rate Limiting	20
2.5.1.1. Forward Proxy / Reverse Proxy	20
2.5.1.2. Распределение нагрузки	20
2.5.1.3. Ограничение частоты запросов	20
2.5.2. Лекция 15: Безопасность и управление доступом	20
2.5.2.1. Аутентификация и авторизация	20
2.5.2.2. RBAC и ABAC	20
2.5.2.3. Single Sign-On	20
2.5.3. Лекция 16: Логирование, мониторинг и планирование задач	20
2.5.3.1. Сбор логов	20
2.5.3.2. Сбор метрик (Prometheus)	20
2.5.3.3. Мониторинг и диагностика (Grafana)	20
2.5.3.4. Планировщики (Celery, Airflow, Dagster)	20
2.6. Модуль 6: System Design	20
2.6.1. Лекция 17: Разбор дизайна конкретной системы	20
2.6.1.1. Применение всех концепций на практике	20
3. Распределённые системы обработки данных	21
3.1. Модуль 1: Компоненты распределённых систем	21
3.1.1. Лекция 1: Основы распределённых систем	21
3.1.1.1. CAP-теорема	21
3.1.1.2. Механизмы репликации данных	21
3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости ..	21
3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций	21
3.1.2.1. Алгоритмы достижения консенсуса	21
3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций	21
3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)	21
3.2. Модуль 2: Распределённые сервисы	21
3.2.1. Лекция 3: Асинхронное взаимодействие сервисов	21
3.2.1.1. Event Bus и Event-Driven Architecture	21

3.2.1.2. Publish-Subscribe vs Message Queue	21
3.2.1.3. Saga Pattern	21
3.2.2. Лекция 4: Kafka	21
3.2.2.1. Внутреннее устройство	21
3.2.3. Лекция 5: Kubernetes	21
3.2.3.1. Роль и место в современных архитектурах	21
3.2.3.2. Области использования	21
3.2.3.3. Внутреннее устройство	21
3.2.4. Лекция 6: ClickHouse	21
3.2.4.1. Архитектура системы хранения и обработки данных	21
3.2.4.2. Партиционирование	21
3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности	22
3.2.5. Лекция 7: Apache Spark	22
3.2.5.1. Роль и место в современных архитектурах	22
3.2.5.2. Области использования	22
3.2.5.3. Внутреннее устройство	22
3.2.5.4. API	22
3.2.6. Лекция 8: Erlang	22
3.2.6.1. Роль и место в современных архитектурах	22
3.2.6.2. Erlang VM (BEAM) и процессы	22
3.2.6.3. Модель акторов	22
3.2.6.4. Fault tolerance и “Let it crash” философия	22
3.2.6.5. OTP фреймворк (Supervisor, GenServer)	22
3.2.6.6. Синхронизация и распределённые вычисления	22
3.2.6.7. Elixir	22
3.3. Модуль 3: Распределенные БД	22
3.3.1. Лекция 9: Большие данные	22
3.3.1.1. Свойства	22
3.3.1.2. Архитектуры	22
3.3.2. Лекция 10: Full Text Search в распределённых системах	22
3.3.2.1. Elasticsearch и Solr	22
3.3.2.2. Распределённый поиск	22
3.3.2.3. Масштабирование	22
3.3.3. Лекция 11: Документные хранилища	23
3.3.3.1. MongoDB: sharding strategies, write concerns	23
3.3.3.2. Object storage: MinIO	23
3.3.4. Лекция 12: Распределенные пространственные и графовые БД	23
3.3.4.1. Графовые БД	23
3.3.4.2. Пространственные БД	23
3.3.4.3. Шардирование графов и пространственных данных	23

1. Глоссарий

1.1. API

Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

1.2. Компонент

Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

1.3. Индекс

Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

1.4. Ввод-вывод

Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

1.5. Слой

Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

2. Архитектура информационных систем

2.1. Модуль 1: Архитектурные основы

2.1.1. Лекция 1: Введение. Принципы построения приложения

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

Рассмотрим несколько примеров:

2.1.1.1. 1. Компонент как класс (C++)

```
#include <iostream>
```

```
class NumberPrinter {
public:
    void print(int x) {
        std::cout << x << std::endl;
    }

    void printRange(int from, int to) {
        for (int i = from; i <= to; ++i) {
            std::cout << i << std::endl;
        }
    }
};
```

Единица вычисления: форматирование и вывод чисел

Интерфейс: сигнатуры публичных методов

Зависимость: консоль ввода-вывода

Форма реализации: класс (ООП-модель)

- Компонент как функция (Elixir)

```
def send_message(host, port, message) do
    {:ok, socket} = :gen_tcp.connect(host, port, [:binary, active: false])
    :ok = :gen_tcp.send(socket, message)
    :gen_tcp.close(socket)
end
```

Единица вычисления: передача сообщения

Интерфейс: сигнатура функции

Зависимость: сеть (TCP stack)

Форма реализации: функция (функциональная модель)

- Компонент как функция (Go)

```
func LoadUserNames(db *sql.DB) ([]string, error) {
    rows, err := db.Query("SELECT name FROM users")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var result []string
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
            return nil, err
        }
    }
}
```

```

    result = append(result, name)
}
return result, nil
}

```

Единица вычисления: извлечение данных

Интерфейс: сигнатура функции

Зависимость: БД (SQL, соединение)

Форма реализации: функция (процедурная / Go-модель)

2.1.1.2. Зависимости компонентов и слоёв

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

Рассмотрим пример слоя персистентности, который сохраняет различные модели в CSV:

```

// domain/models.go - слой моделей данных
package domain

type User    struct{ ID int; Name, Email string }
type Item    struct{ ID int; Title string; Price float64 }
type Storage struct{ ID int; Location string; Capacity int }

// persistence/csv_persistence.go - слой персистентности
package persistence

import (
    "fmt"
    "os"
    "strings"

    "example/domain" // Явная зависимость от слоя моделей
)

func SaveUsersToCSV(users []domain.User, filename string) error {
    lines := make([]string, 0, len(users))
    for _, u := range users {
        lines = append(lines, fmt.Sprintf("%d,%s,%s", u.ID, u.Name, u.Email))
    }
    return write(filename, lines)
}

func SaveItemsToCSV(items []domain.Item, filename string) error {
    lines := make([]string, 0, len(items))
    for _, i := range items {
        lines = append(lines, fmt.Sprintf("%d,%s,%v", i.ID, i.Title, i.Price))
    }
    return write(filename, lines)
}

func SaveStoragesToCSV(storages []domain.Storage, filename string) error {
    lines := make([]string, 0, len(storages))
    for _, s := range storages {
        lines = append(lines, fmt.Sprintf("%d,%s,%d", s.ID, s.Location, s.Capacity))
    }
    return write(filename, lines)
}

func write(filename string, lines []string) error {

```

```

return os.WriteFile(filename, []byte(strings.Join(lines, "\n")), 0644)
}

```

Анализ слоёв:

Слой	Компонент	Контракт (интерфейс)	Зависимости
domain	User	struct{ ID int; Name, Email string }	—
	Item	struct{ ID int; Title string; Price float64 }	—
	Storage	struct{ ID int; Location string; Capacity int }	—
persistence	SaveUsersToCSV	([]domain.User, string) error	domain.User, ФС
	SaveItemsToCSV	([]domain.Item, string) error	domain.Item, ФС
	SaveStoragesToCSV	([]domain.Storage, string) error	domain.Storage, ФС
	write	(string, []string) error	ФС

Заметим:

- **Слой моделей** (domain): три компонента-структуры без зависимостей
- **Слой персистентности** (persistence): четыре компонента-функции, три из которых явно зависят от типов слоя моделей
- Слой персистентности **зависит** от слоя моделей, но не наоборот

2.1.1.3. Продуктивность слоя

Прокси-компонент — компонент, не реализующий собственной вычислительной ответственности, а лишь делегирующий вызовы компонентам других слоёв.

Продуктивность слоя — количественная мера архитектурной самостоятельности слоя, определяемая долей компонентов, реализующих собственную вычислительную ответственность.

Формальное задание меры

Пусть:

- N — общее количество компонентов в слое
- P — количество прокси-компонентов в слое

Тогда коэффициент проксирования слоя определяется как:

$$K_{\text{proxy}} = \frac{P}{N}$$

а продуктивность слоя — как дополняющая величина:

$$K_{\text{prod}} = 1 - K_{\text{proxy}} = 1 - \frac{P}{N}$$

В стандартной литературе (например, Clean Architecture) слой считается продуктивным, если $K_{\text{prod}} \geq 0.8$, то есть не менее 80% компонентов реализуют собственную вычислительную ответственность.

Пример непродуктивного pass-through слоя:

Рассмотрим излишний слой сервисов, который только делегирует вызовы к слою персистентности:

```

// service/user_service.go - непродуктивный слой сервисов
package service

import (
    "example/domain"
    "example/persistence"

```

```

)

func GetUser(id int) (*domain.User, error) {
    return persistence.LoadUser(id)
}

func SaveUser(user *domain.User) error {
    return persistence.SaveUser(user)
}

func DeleteUser(id int) error {
    return persistence.DeleteUser(id)
}

func ListUsers() ([]*domain.User, error) {
    return persistence.ListUsers()
}

```

Анализ слоя сервисов:

Компонент	Контракт	Собственная ответственность
GetUser	(int) (*User, error)	Нет — прямая делегация
SaveUser	(*User) error	Нет — прямая делегация
DeleteUser	(int) error	Нет — прямая делегация
ListUsers	() ([]*User, error)	Нет — прямая делегация

Продуктивность слоя: $K_{\text{prod}} = 1 - \frac{4}{4} = 0$ (0% — полностью непродуктивный слой)

Такой слой не добавляет архитектурной ценности и лишь усложняет кодовую базу, увеличивая количество уровней косвенности без предоставления дополнительной функциональности.

2.1.1.4. Оптимизация системы требует выбора критерия оптимизации

2.1.1.5. Принципы SOLID

2.1.1.6. Dependency Injection

2.1.1.7. Clean Architecture

2.1.2. Лекция 2: Альтернативные архитектурные паттерны

2.1.2.1. Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку.

- “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

2.1.2.2. Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти.

- игровые движки
- обработки физики
- AI
- симуляции

2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC

API¹

2.1.3.1. REST: стандартный веб-протокол

- HTTP
 - методы (GET, POST, PUT, DELETE)
 - коды (Forbidden, Not Found)
- JSON
- URI Endpoints (например, /api/users/123)
- Swagger
- OpenAPI
 - Генерация спецификации из кода
 - Генерация кода из спецификации

2.1.3.2. gRPC: бинарный протокол, высокая производительность

- Историческая справка
 - Вырос из внутренней технологии Google под названием Stubby
 - Использовался для связи между тысячами микросервисов внутри дата-центров
 - В 2015 году открыли исходный код
- IDL (Описание формата общения в .proto-файлах)
- Генерация кода
- Мультиплексирование потоков (Stream Multiplexing)
- Двухнаправленная потоковая передача
- gRPC-Web прокси

¹Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket

2.1.4.1. GraphQL: клиент запрашивает только нужные данные

Клиент формирует запрос с деревом полей, которые он хочет получить

- Проблема необходимости написания десятков различных запросов для получения одной сущности в различных экранах
- Подписка на реальные обновления данных
- Единственная конечная точка (Endpoint)

2.1.4.2. WebSocket: двусторонний канал

- Проблемы Long Polling
- Постоянное соединение, обмен сообщениями в реальном времени в обе стороны.
- Применение
 - Чат-приложения и онлайн-игры.
 - Биржевые тикеры и спортивные результаты.
 - Совместное редактирование документов.
 - Уведомления в реальном времени.

2.2. Модуль 2: Фундаментальные структуры хранилищ данных

Нас беспокоит внутреннее устройство БД для выбора нужного компонента хранения из множества доступных.

2.2.1. Лекция 5: Базовые механизмы хранения данных

2.2.1.1. CSV-файлы как минимальная форма персистентности

CSV-файл — это последовательный текстовый файл, где каждая строка соответствует записи, а поля разделены запятыми (или другим разделителем). Простейшая реализация CRUD над CSV выглядит так:

- Create (вставка новой записи) — $\mathcal{O}(1)$
 - Запись добавляется в конец файла
- Read (поиск записи) — $\mathcal{O}(1)$, n — количество строк
 - Линейный проход по файлу до нужной строки
- Update (обновление записи) — $\mathcal{O}(1)$
 - Перезапись файла полностью
- Delete (удаление записи) — $\mathcal{O}(1)$
 - См Update

2.2.1.2. CSV с добавлением записей и журналированием изменений (WAL, Write Ahead Log)

Внутренняя структура WAL

- Лог — это последовательный файл с бинарными записями формата [TX_ID | OP_TYPE | KEY | OLD_VALUE | NEW_VALUE].
- WAL пишется только последовательно, что делает операции быстрой — запись не требует случайных обращений к диску.
- После подтверждения записи в журнал, изменения асинхронно применяются к CSV.
- Периодически журнал сбрасывается (компрессия): CSV пересоздается с учётом накопленных операций, WAL очищается.
- Create — $\mathcal{O}(1)$
 - Запись операции в конец WAL.
- Read — $\mathcal{O}(n + m)$, m — количество записей в WAL
 - Чтение CSV + применение недоотражённых изменений из журнала
- Update — $\mathcal{O}(1)$
 - Обновление логируется без немедленного изменения CSV
- Delete — $\mathcal{O}(1)$
 - См Update
- Компрессия — $\mathcal{O}(1)$
 - Проигрывание журнала при старте системы или асинхронное применение изменений

Заметим улучшения Update/Delete относительно простого CSV-файла $\mathcal{O}(n) \rightarrow \mathcal{O}(1)$

2.2.1.3. Хеш-индексы как первый уровень ускорения выборки

Хеш-Индекс² создаёт отображение ключ → смещение в файле (offset), что позволяет быстро находить запись в CSV без полного сканирования.

Внутренняя структура:

- Хеш-таблица ($\text{hash}(\text{key}) \rightarrow \text{file_offset}$)
- Смещение указывает на начало строки в CSV, откуда можно считать полную запись

²Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

- Create – $\mathcal{O}(1)$
 - Записывается операция вставки в WAL $\mathcal{O}(1)$
 - Обновление индекса $\mathcal{O}(1)$
- Read – $\mathcal{O}(1)$
 - Вычисляется $\text{hash}(\text{key})$, по индексу извлекается $\{\text{offset}\}$
 - CSV читается напрямую по offset
- Update – $\mathcal{O}(1)$
 - Записывается операция обновления в WAL $\mathcal{O}(1)$
 - Индекс обновляется $\mathcal{O}(1)$
- Delete – $\mathcal{O}(1)$ для индекса, $\mathcal{O}(1)$ при компакссии
 - См Update
- Компакссия – $\mathcal{O}(n + m)$, n – количество строк в CSV, m – количество операций в WAL
 - Проигрываются записи WAL и активные записи из CSV
 - Пересоздаётся новый CSV и индекс только с актуальными (live) записями
 - WAL очищается

Заметим улучшения:

- Операции чтения стали $\mathcal{O}(1)$ за счёт индекса
- Физическая очистка остаётся $\mathcal{O}(1)$, но выполняется редко и асинхронно

Мы справились с проблемой поиска и с проблемой добавления данных. Но заметим что писать один огромный файл - тоже очень плохо. Для решения такой проблемы можно поделить файл на много файлов ограниченного размера. Тогда для каждого файла необходимо будет иметь свой хэш индекса.

2.2.2. Лекция 6: LSM Дерево

2.2.2.1. Общая идея

LSM-дерево решает проблему эффективной записи путём отложенной сортировки и слияния. Вместо немедленной записи на диск в отсортированную структуру, данные буферизуются в памяти и периодически сбрасываются отсортированными порциями.

2.2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)

SSTable — иммутабельный файл с парами ключ-значение, отсортированными по ключу.

Внутренняя структура файла (последовательно):

[Data Block 1][Data Block 2]...[Data Block N][Index Block][Meta Block][Footer]

Data Block:

- Записи: [key_len][key][value_len][value] отсортированные по ключу
- Restart points: массив смещений для быстрого поиска внутри блока (каждые 16 записей)
- Block trailer: [compression_type][crc32]

Index Block:

- Записи формата: [last_key_in_block][block_offset][block_size]
- Позволяет определить нужный блок без чтения всех данных

Meta Block:

- Bloom Filter для всего файла
- Статистика: min/max ключи, количество записей, histogram распределения

Footer (фиксированный размер 48 байт):

- Смещение и размер Index Block
- Смещение и размер Meta Block
- Magic number для валидации формата

Поиск в SSTable — $\mathcal{O}(\log b + k)$, b — количество блоков, k — размер блока

- Бинарный поиск по индексу блоков $\mathcal{O}(\log b)$
- Линейное сканирование внутри блока $\mathcal{O}(k)$

2.2.2.3. Memtable

Memtable — изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Внутренняя структура — сбалансированное дерево (Red-Black Tree или Skip List):

- Поддерживает отсортированный порядок ключей
- При достижении порога размера сбрасывается в новый SSTable
- Вставка в Memtable — $\mathcal{O}(\log m)$, m — количество ключей в Memtable
- Конвертация в SSTable — $\mathcal{O}(m)$, последовательная запись отсортированных данных

Skip List предпочтительнее Red-Black Tree для Memtable поскольку не требует ротаций поддеревьев при вставке

Сложность операций Skip List:

- Insert/Delete/Search — $\mathcal{O}(\log n)$ в среднем, $\mathcal{O}(n)$ в худшем (крайне редко)

2.2.2.4. Bloom Filter

Bloom Filter — вероятностная структура для быстрой проверки отсутствия ключа в SSTable.

Главная идея: избежать дорогих дисковых операций для поиска несуществующих ключей. Вместо чтения полностью всех SSTable с диска, сначала проверяется Bloom Filter в памяти — если он говорит “ключа нет”, SSTable гарантированно не содержит ключ.

Интеграция в SSTable:

- Bloom Filter создаётся при формировании SSTable из Memtable
- Хранится в Meta Block SSTable файла
- При открытии SSTable загружается в память (обычно 10 бит на ключ)
- Проверяется перед любым обращением к данным SSTable

Внутренняя структура:

```
BitArray[m] = [0|1|0|1|1|0|1|0|...] // m бит
HashFunctions[k] = [h1, h2, ..., hk] // k независимых хеш-функций
```

Параметры:

- m — размер битового массива
- k — количество хеш-функций
- n — ожидаемое количество элементов

Алгоритм добавления элемента x:

```
for i = 1 to k:
    index = hi(x) mod m
    BitArray[index] = 1
```

Алгоритм проверки элемента x:

```
for i = 1 to k:
    index = hi(x) mod m
    if BitArray[index] == 0:
        return NOT_PRESENT // гарантированно отсутствует
return MAYBE_PRESENT // возможно присутствует
```

- Проверка ключа — $\mathcal{O}(k)$, k — количество хеш-функций (обычно 3-10)
- Добавление ключа — $\mathcal{O}(k)$
- False positive rate: $(1 - e^{\{-k \frac{n}{m}\}})^k$ при случайном распределении
- False negative rate: 0 (если Bloom Filter говорит “нет”, ключа точно нет)
- Память: m бит независимо от размера ключей

Экономия ресурсов:

- Без Bloom Filter: чтение Index Block + Data Block для каждого отсутствующего ключа
- С Bloom Filter: только проверка в памяти для 99%+ отсутствующих ключей (при p=1%)

2.2.2.5. Компакция – причины, уровни

Компакция объединяет несколько SSTable файлов в один для:

- Уменьшения числа файлов (улучшение производительности чтения)
- Удаления устаревших версий данных (delete операции, перезаписи)
- Уменьшения размера базы данных (удаление дубликатов)

Концепция уровней: LSM использует иерархию уровней, каждый уровень содержит отсортированные данные:

- L_0 : несортированные SSTable, пришедшие из Memtable (ключи **пересекаются**)
- L_1 : отсортированные SSTable, ключи **не пересекаются** внутри уровня
- $L_2, L_3, \dots, L_n$: каждый уровень в 10 раз больше предыдущего

Размер уровней:

- L_0 : до 4 файлов (типичный порог), каждый 2 МБ
- L_1 : 20 МБ ($10 \times$ размер L_0)
- L_i : 10^i МБ

2.2.2.6. Критерии запуска компакций

Компакция запускается когда:

1. L_0 достигает порога (обычно 4 файла) $\rightarrow$ компакция $L_0 \rightarrow L_1$
2. Размер уровня L_i превышает лимит $\rightarrow$ компакция $L_i \rightarrow L_{i+1}$
3. Поиск затрагивает слишком много файлов на одном уровне $\rightarrow$ принудительная компакция

2.2.2.7. Что происходит при компакциях

Компакция $L_0 \rightarrow L_1$:

- Читаем несортированные файлы из L_0 (возможны пересечения ключей)
- Читаем перекрывающиеся данные из L_1 (может быть много файлов)
- Выполняем k-way merge (слияние k отсортированных последовательностей)
- Удаляем старые версии данных во время слияния
- Записываем результат в L_1
- Сложность: $\mathcal{O}(N)$ где N — размер всех данных, значительная Ввод-вывод³ нагрузка

Компакция $L_i \rightarrow L_{i+1}$ ($i \geq 1$):

- Выбираем файлы из L_i , пересекающиеся по ключам с L_{i+1}
- Выполняем 2-way merge: отсортированные данные из L_i + перекрывающиеся из L_{i+1}
- Записываем результат в L_{i+1}
- **Сложность: $\mathcal{O}(S)$ где S — размер обрабатываемых данных (очень эффективно, т.к. ключи отсортированы)**

2.2.2.8. Почему L_0 специальный?

L_0 — единственный уровень, где ключи **пересекаются**. Это делает компакцию L_0 дорогой:

- Нужно проверить все файлы L_0 друг против друга
- Нужно слить с **потенциально всеми** файлами L_1 (так как ключи могут быть везде)
- Поэтому ограничивают число файлов в L_0 (например, max 4 файла)

На уровнях L_1 и выше ключи не пересекаются внутри уровня, что позволяет эффективно сливать только перекрывающиеся по ключам диапазоны.

2.2.2.9. Tombstones

Tombstone — специальная запись-маркер удаления, так как SSTable иммутабельны.

- Delete — $\mathcal{O}(\log m)$, запись tombstone в Memtable
- Физическое удаление происходит при компакции, когда tombstone достигает последнего уровня

2.2.2.10. Полная структура LSM-дерева

Архитектура компонентов:

1. WAL для восстановления Memtable после сбоя
2. Активный Memtable для записи
3. Иммутабельные Memtable, ожидающие сброса
4. Множественные уровни SSTable с Bloom Filter для каждого

CRUD операции:

- Create (вставка новой записи) — $\mathcal{O}(\log m)$, m — размер Memtable
 - Запись в WAL $\mathcal{O}(1)$ — последовательная запись в конец файла
 - Вставка в Memtable (Skip List) $\mathcal{O}(\log m)$
 - При переполнении Memtable становится иммутабельным, создается новый
- Read (поиск записи) — $\mathcal{O}(\log m + L(k + \log b))$, L — количество уровней, m — размер Memtable, b — кол-во блоков в SSTable
 - Поиск в активном Memtable $\mathcal{O}(\log m)$
 - Поиск в иммутабельных Memtable $\mathcal{O}(\log m)$ для каждого
 - Для каждого уровня SSTable:
 - Проверка Bloom Filter $\mathcal{O}(k)$, k — количество хеш-функций
 - Если Bloom положительный: бинарный поиск в Index Block $\mathcal{O}(\log b)$ + чтение Data Block
 - Возврат первого найденного значения (свежие данные приоритетнее)

³Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

- Update (обновление записи) – $\mathcal{O}(\log m)$
 - Идентично Create — новая версия записывается в Memtable
 - Старые версии остаются в SStable до компаккции
 - При чтении возвращается самая свежая версия
- Delete (удаление записи) – $\mathcal{O}(\log m)$
 - Запись tombstone в WAL $\mathcal{O}(1)$
 - Вставка tombstone в Memtable $\mathcal{O}(\log m)$
 - Физическое удаление при компаккции когда tombstone достигает последнего уровня
- Range Query (диапазонный запрос) – $\mathcal{O}(\log m + L \cdot \log b + R)$, R — размер результата
 - Создание итераторов для Memtable и каждого SStable
 - Merge итераторов с учётом версионности
 - SStable уже отсортированы — эффективное слияние

2.2.3. Лекция 7: В-дерево: индексирование на диске

В-дерево решает проблему эффективного использования кэша диска и минимизации случайных обращений путём группировки данных в узлы фиксированного размера, соответствующие блокам диска.

2.2.3.1. В-фактор и структура узла

В-фактор — это максимальное количество указателей на детей в одном узле (или эквивалентно, максимальное количество ключей + 1).

Внутренняя структура узла порядка B :

- Ключи: $[k_1, k_2, \dots, k_{B-1}]$ в отсортированном порядке (максимум $B - 1$ ключей)
- Указатели на детей: $[p_0, p_1, \dots, p_B]$ где p_i указывает на поддерево с ключами в диапазоне $[k_i, k_{i+1})$
- Листовой узел содержит значения вместо указателей на поддеревья
- Каждый узел занимает ровно одно дисковое обращение — это ключевая идея

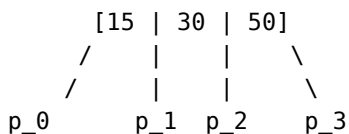
Выбор В-фактора:

- В выбирается так, чтобы узел точно поместился в один блок диска
- Если блок диска = 4 КБ, а каждая пара (ключ, указатель) занимает 10 байт, то $B \approx 400 - 500$
- Маленький B (например, $B=4$): много уровней дерева, много обращений к диску
- Большой B (например, $B=500$): глубина дерева $\mathcal{O}(\log_B n)$ очень мала (для 1 млн записей при $B=500$ глубина всего 3-4 уровня), но поиск внутри узла медленнее

Инвариант В-дерева:

- Все листья находятся на одной глубине h (в отличие от AVL, где глубина может отличаться на 1)
- Каждый внутренний узел содержит от $\lceil \frac{B}{2} \rceil$ до $B - 1$ ключей
- Корень содержит минимум 1 ключ

Пример узла В-дерева при $B=4$:



p_0: ключи < 15
p_1: ключи в [15, 30)
p_2: ключи в [30, 50)
p_3: ключи >= 50

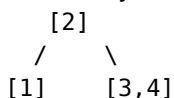
Пример роста В-дерева порядка $B=3$ при вставке 1,2,3,4,5:

Вставляем 1: [1]

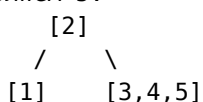
Вставляем 2: [1,2]

Вставляем 3: [1,2,3]

Вставляем 4: узел переполнен! Разделяем:



Вставляем 5:



При вставке нового элемента в переполненный узел:

- Узел разделяется на две половины
- Медиана поднимается в родителя

- Если родитель тоже переполнен, рекурсивно разделяем его
- Это может привести к вырастанию корня и увеличению высоты дерева на 1
- Create – $\mathcal{O}(\log_B n)$
 - Поиск листового узла $\mathcal{O}(\log_B n)$ дисковых обращений
 - Вставка ключа-значения в лист $\mathcal{O}(1)$
 - При переполнении узла: разделение на две половины и поднятие медианы $\mathcal{O}(\log_B n)$ обращений вверх по дереву
- Read – $\mathcal{O}(\log_B n)$
 - Бинарный поиск по ключам текущего узла $\mathcal{O}(\log B)$ — в памяти, быстро
 - Переход к дочернему узлу через одно дисковое обращение
 - Повторение до листа: максимум $\mathcal{O}(\log_B n)$ обращений
- Update – $\mathcal{O}(\log_B n)$
 - Поиск записи $\mathcal{O}(\log_B n)$
 - Обновление значения in-place (B-дерево не версионировано, в отличие от LSM)
- Delete – $\mathcal{O}(\log_B n)$
 - Поиск и удаление ключа $\mathcal{O}(\log_B n)$
 - При недостаточном количестве ключей: слияние с соседним узлом или перемещение ключей из соседа
- Range Query – $\mathcal{O}(\log_B n + \frac{R}{B})$, R — размер результата
 - Поиск левой границы $\mathcal{O}(\log_B n)$
 - Последовательное сканирование листьев (один узел = один читаемый блок диска)
 - Эффективно благодаря локальности: листья часто находятся рядом на диске

Преимущества относительно LSM:

- Нет компакций, стабильная задержка
- Меньше I/O амортизировано: B больше, чем типичный размер блока

Недостатки:

- Запись требует read-modify-write: чтение узла → изменение → запись обратно
- Сложность балансировки при вставке и удалении
- Для частых записей LSM может затратить сильно меньше Ввод-вывод⁴

⁴Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

2.2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища

2.2.4.1. ACID (Atomicity, Consistency, Isolation, Durability)

2.2.4.2. Уровни изоляции

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

2.2.4.3. Аналитические базы данных (OLAP) и колоночные хранилища

2.2.4.4. Сжатие колонок

2.2.4.5. Сравнение подходов

2.3. Модуль 3: Другие хранилища данных

2.3.1. Лекция 9: Аналитика в оперативной памяти

2.3.1.1. Vectorized execution (блочная обработка)

2.3.1.2. DuckDB

- Чтение из различных источников
- Трансформация данных
- Запись

2.3.2. Лекция 10: Кэш в оперативной памяти

2.3.2.1. Skip List

2.3.2.2. Incremental Rehashing

2.3.2.3. HyperLogLog

2.3.2.4. Redis

- Архитектура (однопоточность, event-driven)

2.4. Модуль 4: Методы поиска похожих/близких элементов

2.4.1. Лекция 11: Специализированные индексы для поиска

2.4.1.1. Inverted Index (перевёрнутые индексы)

- Постинг-листы (posting lists)
- Применение: Elasticsearch, Lucene

2.4.1.2. Trie (префиксные деревья)

- Автодополнение, префиксный поиск

2.4.1.3. Bitmap indexes

- Для категориальных данных

2.4.2. Лекция 12: Векторные индексы для поиска подобия

2.4.2.1. Основы векторных представлений

2.4.2.2. HNSW (Hierarchical Navigable Small World)

- Граф как индекс
- Поиск соседей

2.4.2.3. LSH (Locality-Sensitive Hashing)

- Вероятностный поиск

2.4.2.4. Примеры: Weaviate, Pinecone, Qdrant

2.4.3. Лекция 13: Геопространственные структуры

2.4.3.1. R-Tree

- Bounding boxes, MBR
- Применение: PostGIS

2.4.3.2. QuadTree и KD-Tree

- Разбиение пространства
- Когда какое дерево эффективнее

2.5. Модуль 5: Операционные компоненты

2.5.1. Лекция 14: Прокси и Rate Limiting

2.5.1.1. Forward Proxy / Reverse Proxy

2.5.1.2. Распределение нагрузки

- Round Robin
- Least Connections
- Weighted Least Connections
- IP Hash
- Consistent Hashing

2.5.1.3. Ограничение частоты запросов

- Token Bucket
- Sliding Window

2.5.2. Лекция 15: Безопасность и управление доступом

2.5.2.1. Аутентификация и авторизация

2.5.2.2. RBAC и ABAC

2.5.2.3. Single Sign-On

2.5.3. Лекция 16: Логирование, мониторинг и планирование задач

2.5.3.1. Сбор логов

2.5.3.2. Сбор метрик (Prometheus)

2.5.3.3. Мониторинг и диагностика (Grafana)

2.5.3.4. Планировщики (Celery, Airflow, Dagster)

2.6. Модуль 6: System Design

2.6.1. Лекция 17: Разбор дизайна конкретной системы

2.6.1.1. Применение всех концепций на практике

3. Распределённые системы обработки данных

3.1. Модуль 1: Компоненты распределённых систем

3.1.1. Лекция 1: Основы распределённых систем

3.1.1.1. CAP-теорема

- Следствия компромиссов между согласованностью, доступностью и устойчивостью к разделению сети
- Примеры CA, AP, CP систем

3.1.1.2. Механизмы репликации данных

- Архитектура “ведущий-подчинённый” (Leader-Follower)
- Репликация с использованием кворумов (quorum-based replication)

3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости

- Механизмы решения классических проблем распределённых систем
- Применение в системах Kafka и ClickHouse

3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций

3.1.2.1. Алгоритмы достижения консенсуса

- Raft
- Paxos

3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций

3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)

- Этапы выполнения
- Гарантии надёжности

3.2. Модуль 2: Распределённые сервисы

3.2.1. Лекция 3: Асинхронное взаимодействие сервисов

3.2.1.1. Event Bus и Event-Driven Architecture

3.2.1.2. Publish-Subscribe vs Message Queue

3.2.1.3. Saga Pattern

3.2.2. Лекция 4: Kafka

3.2.2.1. Внутреннее устройство

3.2.3. Лекция 5: Kubernetes

3.2.3.1. Роль и место в современных архитектурах

3.2.3.2. Области использования

3.2.3.3. Внутреннее устройство

3.2.4. Лекция 6: ClickHouse

3.2.4.1. Архитектура системы хранения и обработки данных

- Принципы организации хранения
- Методы компрессии данных

3.2.4.2. Партиционирование

- Выбор ключа
- Управление партициями (добавление, удаление)

3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности

- Базовая таблица MergeTree и её специализированные варианты
- ReplacingMergeTree для управления версионированием
- ReplicatedMergeTree для обеспечения отказоустойчивости

3.2.5. Лекция 7: Apache Spark

3.2.5.1. Роль и место в современных архитектурах

3.2.5.2. Области использования

3.2.5.3. Внутреннее устройство

3.2.5.4. API

- Dataframe
- SparkSQL

3.2.6. Лекция 8: Erlang

3.2.6.1. Роль и место в современных архитектурах

3.2.6.2. Erlang VM (BEAM) и процессы

3.2.6.3. Модель акторов

3.2.6.4. Fault tolerance и “Let it crash” философия

3.2.6.5. OTP фреймворк (Supervisor, GenServer)

3.2.6.6. Синхронизация и распределённые вычисления

3.2.6.7. Elixir

3.3. Модуль 3: Распределенные БД

3.3.1. Лекция 9: Большие данные

3.3.1.1. Свойства

- Объем
- Скорость
- Разнообразие
- Достоверность

3.3.1.2. Архитектуры

- Modern Data Architecture
- Lambda
- Lakehouse

3.3.2. Лекция 10: Full Text Search в распределённых системах

3.3.2.1. Elasticsearch и Solr

- Sharding: hash-based, range-based
- Replication и replica factor
- Eventual consistency при индексировании

3.3.2.2. Распределённый поиск

- Broadcast query, merge результатов
- Search After вместо offset/limit

3.3.2.3. Масштабирование

- Hot shards проблема

- Index rollover, segment merging

3.3.3. Лекция 11: Документные хранилища

3.3.3.1. MongoDB: sharding strategies, write concerns

3.3.3.2. Object storage: MinIO

3.3.4. Лекция 12: Распределенные пространственные и графовые БД

3.3.4.1. Графовые БД

3.3.4.2. Пространственные БД

3.3.4.3. Шардирование графов и пространственных данных